CS 550 Operating Systems, Spring 2026
Programming Project 1 (PROJ1)

Out: 3/7/2026, SAT
Due date: 4/4/2026, SAT 23:59:59

In this project, you will implement an enhanced version of `geptpid()`, allowing the caller to obtain the PID of any process in the direct lineage of the process family tree. This project will help you understand how system calls are implemented in modern operating systems and how they are invoked from a user program in xv6.

This project contributes 5% toward the final grading.

1 GitHub account username

The projects in this semester will be administered collectively using GitHub Classroom and Brightspace. Complete the following Google form to inform the instructor of the GitHub account you will use for the projects. If your GitHub username is not submitted before grading starts, 5 points will be deducted from your final score for this project.

Form link: `https://forms.gle/wB9XJKC9bek25QRF6`

(Continue to the next page.)

2 Baseline source code

You will work on the base code that needs to be cloned/downloaded from your own private GitHub repository. Make sure you read this whole section, as well as the grading guidelines (Section 6), before going to the following link at the end of this section.

- Go to the link at the end of this section to accept the assignment.
- Work on and commit your code to the default branch of your repository. Do not create a new branch. Failure to do so will lead to problems with the grading script and 5 points off of your project grade.

Assignment link: `https://classroom.github.com/a/jY9VzLmG`

(Continue to the next page.)

3 Build and run xv6

The following provides instructions on building and running xv6 using either a CS machine or your computer.

(1) Log into a CS machine (i.e., a local machine or a remote cluster machine).

(2) Clone or download the baseline xv6 code.

(3) Compile and run xv6:

```
$ make qemu-nox
```

After compiling the kernel source and generating the root file system, the Makefile will start a QEMU VM to run the xv6 kernel image just compiled (you can read the Makefile for more details). Then, you will see an output similar to the following, indicating you have successfully compiled and run the xv6 system.

```
Booting from Hard Disk..xv6...
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap start 58
init: starting sh
$
```

Troubleshooting:

- If you see the following error message:

```
make: execvp: ./sign.pl: Permission denied
```

  Solve it by running the following command:

```
chmod ugo+x ./sign.pl
```

  Then

```
make clean
make qemu-nox
```

- If you get a "No bootable device" error message, make sure you "make clean" before re-building the OS using "make qemu-nox".

(Continue to the next page.)

4 Coding: enabling a "super" version of `getpid()` (64 points)

4.1 `getpid_super()` and the corresponding system call

In xv6 (and almost all other operating systems), a user process can call the C function getpid() to obtain its process ID (PID). The `getpid()` function's prototype is:

```
pid_t getpid(void);
```

Explanation:

- `pid_t`: A data type used for process IDs (typically an integer).
- `getpid()`: Returns the Process ID (PID) of the calling process.
- `void`: Indicates that the function does not take any parameters.

The coding task in this project is to implement a "super" version of the `getpid()` function that allows the calling process to obtain the PID of any process in the direct lineage of the process family tree. The prototype of this new function is:

```
int getpid_super(int level_offset, int * pid_ptr);
```

Explanation:

- `getpid_super()` allows the calling process to obtain the PID of the process at the specified `level_offset` within its direct lineage in the process family tree.
- The first argument, `level_offset`, specifies the hierarchical level offset of the process whose PID the calling process wants to retrieve.
  - Setting `level_offset` to 0 means the calling process intends to obtain its own PID, equivalent to calling `getpid()`.
  - Setting `level_offset` to a negative integer indicates that the calling process wants to obtain the PID of one of its ancestor processes within its direct lineage in the process family tree.

    For example, setting `level_offset` to -1 means the calling process wants the PID of its parent, -2 refers to its grandparent, and -3 corresponds to its great-grandparent, and so on.
  - Setting `level_offset` to a positive integer indicates that the calling process wants to obtain the PID of one of its offspring processes within its direct lineage in the process family tree. For simplicity, you can assume that each process can have at most one child process.

    For example, setting `level_offset` to 1 means the calling process wants the PID of its child, 2 refers to its grandchild, and 3 corresponds to its great-grandchild, and so on.

- Upon a successful return from the `getpid_super()` function, the PID of the requested process is stored in the memory location pointed to by the second argument, `pid_ptr`. This pointer is provided by the calling process when invoking `getpid_super()`.
- The return value of `getpid_super()` can be one of the following:
  - **0**: The requested PID was successfully obtained.
  - **-1**: The requested PID does not correspond to an existing process.
  - **-2**: The memory location pointed to by the second argument, `pid_ptr`, is invalid (e.g., it is 0).

You learned in class that examining process information and returning the requested data are privileged operations that should be handled by the OS kernel. Using the terminology from class, the `getpid_super()` function is a user-level "library wrapper function" for the corresponding system call, which performs the actual work at the kernel level.

In this project, your tasks are as follows:

- Implement the user-level library function `getpid_super()`.
- Implement the corresponding system call that performs the actual work as described above.
- Design and develop a test program to verify your implementation.

When working on the coding and testing, keep the following in mind:

- Your implementation must strictly follow the exact prototype of `getpid_super()` as described above. Any deviation will result in failed test cases in our grading test program.
- You can name the system call however you like since the user program (i.e., the test program) does not directly interact with or recognize the system call.
- Your test program will not be evaluated or graded, as we will use our own test program for grading. However, you do not need to remove your test program from your submission. You may leave it in your code, but it will not be used.

4.2 Hints

- Reading and understanding how existing user commands/programs and their associated system calls are implemented will be helpful. For example, you can examine how the `cat` user command is implemented. The `cat` command is defined in `cat.c`, where it calls the `open()` system call. The actual implementation of the `open()` system call is handled in the `sys_open()` function in `sysfile.c`.
- Understanding the underlying mechanisms is crucial. Pay attention to all relevant files, including assembly files. (Don't worry, the relevant assembly file is straightforward to read.)

- Process-related functionalities in the kernel are implemented in `proc.c`. Carefully reviewing this file will be helpful.
- In your test program, you need to create a chain of processes with a parent-child relationship, not a sibling relationship. This means you should call the `fork()` function within child processes to create the chain.
- If you use a `for` loop to create processes in your test program, ensure your code logic avoids the "fork bomb" problem. A fork bomb occurs when both parent and child processes call `fork()` iteratively, causing the number of processes to grow exponentially.
- In your test program, using `sleep()` to pause a process is preferable to calling `wait()`. This is because when `wait()` returns, the targeted child process has already exited.
  - In xv6, the `sleep(n)` function pauses the execution of a process for `n` clock ticks. Since xv6 uses a 100 Hz timer (100 timer interrupts per second), each tick corresponds to 10 milliseconds of wall-clock time. Therefore, calling `sleep(100)` will cause the process to sleep for approximately 1 second of wall-clock time.
- **Test your code thoroughly on a CS machine before submitting.**

4.3 Submit your code

Once your code in your GitHub private repository is ready for grading, submit a text file named "DONE" (along with the "xv6-syscall-mechanisms.pdf" detailed in the next section) to the assignment link in Brightspace.

We cannot determine if your code is ready for grading in your GitHub repository until we see the "DONE" file in Brightspace. Failure to submit the "DONE" file will result in a late penalty, as specified in the "Grading" section.

(Continue to the next page.)

5 xv6 system call implementation Q&A (36 points)

Answer the following questions about system call implementation.

(1) (9 points) In which file is the user-level library wrapper function `int getpid_super(int level_offset, int * pid_ptr)` defined?

(2) (9 points) Explain (with the actual code) how the above wrapper function triggers the system call.

(3) (9 points) Explain (with the actual code) how the OS kernel locates a system call and calls it (i.e., the kernel-level operations of calling a system call).

(4) (9 points) How are arguments of a system call passed from user space to OS kernel?

Key in your answers to the above questions with the editor you prefer, export them in a PDF file named "xv6-syscall-mechanisms.pdf", and submit the file to the assignment link in Brightspace.

Last but not least, if you have referenced any online materials or resources (including code and Q&A) while completing this project, you must list all references in the "DONE" file. Failure to do so, if detected, will result in a zero for the entire project and may lead to additional penalties depending on the severity of the violation.

6 Grading

The following are the general grading guidelines for this and all future projects.

(1) **The code in your repository will not be graded until a "DONE" file is submitted to Brightspace**.

(2) The submission time of the "DONE" file shown on the Brightspace system will be used to determine if your submission is on time or to calculate the number of late days. Late penalty is 10% of the points scored for each of the first two days late, and 20% for each of the days thereafter.

(3) If you are to compile and run the xv6 system on the department's remote cluster, remember to use the baseline xv6 source code provided by our GitHub classroom. Compiling and running xv6 source code downloaded elsewhere can cause 100% CPU utilization on QEMU.

Removing the patch code from the baseline code will also cause the same problem. So make sure you understand the code before deleting them.

If you are reported by the system administrator to be running QEMU with 100% CPU utilization on QEMU, 10 points off.

(4) If the submitted patch cannot successfully patched to the baseline source code, or the patched code does not compile:

```
1   TA will try to fix the problem (for no more than 3 minutes);
2   if (problem solved)
3     1%-10% points off (based on how complex the fix is, TA's discretion);
4   else
5     TA may contact the student by email or schedule a demo to fix the problem;
6     if (problem solved)
7       11%-20% points off (based on how complex the fix is, TA's discretion);
8     else
9       All points off;
```

So in the case that TA contacts you to fix a problem, please respond to TA's email promptly or show up at the demo appointment on time; otherwise the line 9 above will be effective.

(5) If the code is not working as required in the project spec, the TA should take points based on the assigned full points of the task and the actual problem.

(6) **Lastly but not the least, stick to the collaboration policy stated in the syllabus: you may discuss with you fellow students, but code should absolutely be kept private. Any kind of cheating will result in zero point on the project, and further reporting.**